COMP232 Lab 2

Encryption in Java or Python

In this lab session, you will study some simple Java or Python programs implementing DES encryption/decryption with standard JCA/JCE libraries.

> **Note:**
>
> No specific IDE or other development tools are assumed. All exercises can be done using simple editors and command line for compiling and executing programs.

> **References:**
>
> JCA/JCE Reference Manual
> DES Encryption in Java
> DES Encryption in Python

Note: If you are using the Python version, please install the `pycryptodome` package using: `pip install pycryptodome` for this lab.

1. DES Encryption

Compile and run the programs for the following:

- DES encryption in ECB mode.
- DES encryption in CBC mode with an inline IV.
- DES encryption in CBC mode with IV generated by Cipher object.

Look into the code and see how the encryption is implemented.

Task 1

Using the programs above (and their modifications) demonstrate the advantage of CBC mode over ECB mode.

2. Password-Based Encryption (PBE)

In the examples of programs given in the previous section, the keys used in encryption were generated automatically without any user input. In many cases we would like to implement password-based encryption.

Password-based encryption mechanisms in JCE are based on cryptographic *hashing* mechanisms. In short, a password and a salt, which is a random data is fed into a mixing

function (a kind of hash function) which is applied iteratively the number of times defined by iteration count. All this happens in the *Secret Key Factory* object. The result is used then to create a key in the Cipher object. To decrypt one has to know the password (secret piece of data), the salt and iteration count (these two are usually not considered as secret and may be known to the attacker).

Task 2

Java Version

- Download the Password-based encryption program `PBEs.java` and auxiliary utility program `Util.java`.
- Compile both programs (e.g. `> javac *.java`), execute PBEs and explore the code to find out how to do password-based encryption.
- Try then to change iteration count to a smaller value and see how this affects the execution time.
- Modify the program, so it can decrypt the encrypted message.

Implementation of decryption is very similar to the case of encryption. You need to do the same preparatory steps as for encryption (including salt and iteration count initialization, creation PBE parameter set, initialization of password, etc).
The only difference is that initialization of PBE Cipher should be done differently, as shown in the fragment below:

```
// Initialize PBE Cipher with key and parameters
pbeCipher.init(Cipher.DECRYPT_MODE, pbeKey, pbeParamSpec);

// Decrypt the ciphertext
byte[] plaintext = pbeCipher.doFinal(aCyphertext);
String StringPlaintext = new String (plaintext);
```

Python Version

- Download the Password-based encryption program `PBE.py`
- Run the script using `python3 PBE.py`
- Try then to change iteration count to a smaller value and see how this affects the execution time.
- Modify the program, so it can decrypt the encrypted message.

3. After the Lab: from DES to AES

Once you have tried all the above, try to repeat Task 1 above with AES rather than DES encryption algorithm. Essentially you will need to replace ”DES” with ”AES” everywhere.

Java Version

In the case of difficulties please consult the Reference Manual of JCA/JCE, which contains fragments of code illustrating the use of AES. Please notice that password-based encryption for AES implemented in JCA/JCE is limited to the key size 128 bits.

You can further explore the sample programs `PBE_AES.java` and `PBE_AES2.java` for password-based AES encryption.

Python Version

In the case of difficulties please consult the Reference Manual of Python, which contains fragments of code illustrating the use of AES. You can explore PBEs using AES with different key size.